



CPE223 Computer Architectures Mini-Project Report

Project Title:

Modified SAP-1

Submitted by

Guntee Doungmanee	67070501003
Natthawat Primsirikunawut	67070501027
Wisit Suwannao	67070501042
Supawit Marayat	67070501045
Polwarit Watthanahemmarat	67070501067

Submitted to

Asst. Prof. Rajchawit Sarochawikasit

Department of Computer Engineering

King Mongkut's University of Technology Thonburi

Semester 2/2025

Contents

1	Introduction	2
2	System Architecture and Datapath Design	2
3	Module Descriptions	4
3.1	Program Counter	4
3.2	Memory Address Register	4
3.3	RAM	5
3.4	Instruction Register	5
3.5	Accumulator	5
3.6	B Register	6
3.7	ALU	6
3.8	Output Register	6
3.9	Control Unit	7
3.10	Bus Multiplexer	7
4	Instruction Set and Opcode Format	8
5	ALU Design	9
5.1	ADD and SUB	9
5.2	Multiplication	10
5.3	Division	11
5.3.1	Division by Zero Handling	12
5.4	Opcode-Based Result Selection	12
6	Control Unit and FSM Design	13
7	Simulation Results	14
7.1	Main Program	14
7.2	Waveform Walk-Through	15
7.3	Divide-by-Zero Result	15
7.4	Test Summary	15
8	Conclusion	16
9	References	17

1 Introduction

SAP-1, or Simple-As-Possible-1, is a minimal educational computer architecture used to demonstrate how a processor fetches instructions, decodes them, and executes them through a controlled datapath. The design in this project implements a modified SAP-1 computer using Verilog HDL. It includes the standard SAP-1 style components: program counter, memory address register, RAM, instruction register, accumulator, B register, ALU, output register, shared bus, and controller-sequencer.

The project extends the basic SAP-1 arithmetic behavior by adding multiplication and division. The multiplication and division logic is implemented inside the ALU from shifts, additions, comparisons, and subtractions. The MUL operation does not use the Verilog `*` operator, and the DIV operation does not use the Verilog `/` operator.

2 System Architecture and Datapath Design

The computer is organized as an 8-bit multi-cycle processor with a 4-bit address space. RAM contains 16 locations, and each location stores one 8-bit value. A RAM word may represent either an instruction or a data value. Instructions are fetched from RAM using the program counter and memory address register, then stored in the instruction register before execution.

Component	Verilog File	Purpose
Program Counter	<code>program_counter.v</code>	Stores the address of the next instruction.
Memory Address Register	<code>memory_address_register.v</code>	Stores the RAM address currently being accessed.
RAM	<code>ram_16x8.v</code>	Stores program instructions and data values.
Instruction Register	<code>instruction_register.v</code>	Stores the fetched instruction and separates opcode and operand.
Accumulator	<code>accumulator.v</code>	Main working register for arithmetic and load operations.
B Register	<code>b_register.v</code>	Holds the second operand for ALU operations.
ALU	<code>alu.v</code>	Performs ADD, SUB, MUL, and DIV.
Output Register	<code>output_register.v</code>	Stores the value produced by the OUT instruction.
Bus Multiplexer	<code>bus_mux.v</code>	Selects one datapath source onto the internal bus.
Control Unit	<code>control_unit.v</code>	FSM that generates register load, bus select, PC increment, ALU opcode, and halt signals.

Component	Verilog File	Purpose
Top Module	sap1_top.v	Connects the full CPU datapath, control unit, RAM programming interface, and external outputs.

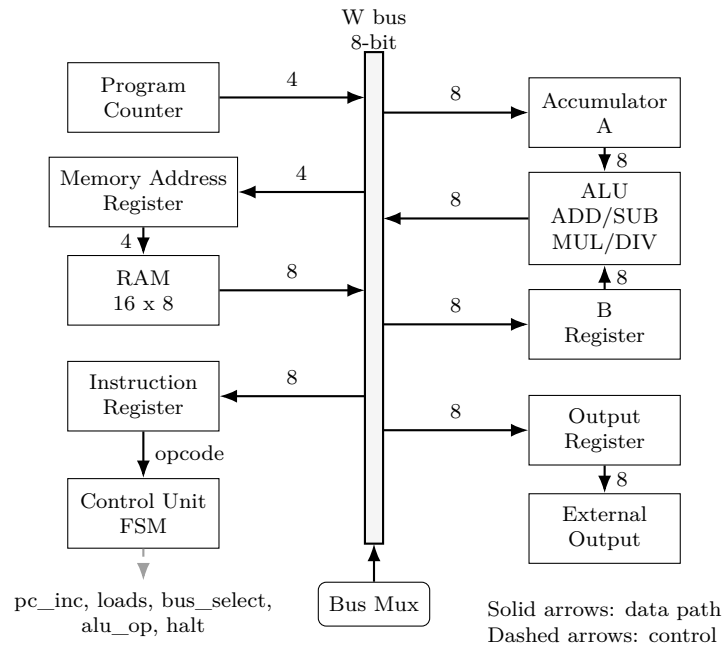


Figure 1: Top-level datapath of the modified SAP-1 processor.

The main datapath widths are shown below.

Signal or Storage	Width	Description
Internal data bus	8 bits	Moves data and instructions between modules.
RAM word	8 bits	Stores one instruction or one data value.
RAM address	4 bits	Selects one of 16 memory locations.
Opcode	4 bits	Upper nibble of the instruction.
Operand	4 bits	Lower nibble of the instruction.
Program counter	4 bits	Stores the next instruction address.

The shared bus is implemented using a multiplexer. This preserves the single-bus SAP-1 behavior while avoiding internal tri-state wiring, which is not a good target for FPGA synthesis and can create multiple-driver problems if control signals overlap. Only one source is selected onto the bus at a time.

Bus Source	Value Placed on Bus
BUS_ZERO	8'h00
BUS_PC	{4'h0, pc_data}
BUS_RAM	RAM output data
BUS_OPERAND	{4'h0, ir_operand}
BUS_ACC	Accumulator output
BUS_ALU	ALU output

3 Module Descriptions

3.1 Program Counter

The Program Counter in `program_counter.v` is a 4-bit register that points to the next instruction in RAM. During `FETCH_ADDR`, the control unit selects the PC onto the bus so the memory address register can capture it. During `FETCH_INST`, the control unit increments the PC so the next instruction address is ready for the following fetch.

```

1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         pc_out <= 4'h0;
4     else if (load)
5         pc_out <= data_in;
6     else if (increment)
7         pc_out <= pc_out + 4'h1;
8 end

```

Listing 1: Program Counter update logic from `program_counter.v`

3.2 Memory Address Register

The Memory Address Register, or MAR, in `memory_address_register.v` stores the 4-bit RAM address currently being used. It receives the PC value during instruction fetch and receives the instruction operand during memory-reference instructions such as LDA, ADD, SUB, MUL, and DIV.

```

1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         address <= 4'h0;
4     else if (load)
5         address <= data_in;
6 end

```

Listing 2: MAR load logic from `memory_address_register.v`

3.3 RAM

The RAM in `ram_16x8.v` contains 16 8-bit words. The CPU reads memory asynchronously using the current MAR address. The testbench and top-level programming interface load program bytes and data bytes before reset is released by asserting `prog_we`.

```
1 reg [7:0] memory [0:15];
2
3 assign data_out = memory[address];
4
5 always @(posedge clk) begin
6     if (prog_we)
7         memory[prog_addr] <= prog_data;
8 end
```

Listing 3: RAM read and program loading from `ram_16x8.v`

3.4 Instruction Register

The Instruction Register in `instruction_register.v` captures the instruction fetched from RAM. The upper nibble becomes the opcode sent to the control unit and ALU, while the lower nibble becomes the operand address used for memory-reference instructions.

```
1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         instruction <= 8'h00;
4     else if (load)
5         instruction <= data_in;
6 end
7
8 assign opcode = instruction[7:4];
9 assign operand = instruction[3:0];
```

Listing 4: Instruction load and field extraction from `instruction_register.v`

3.5 Accumulator

The Accumulator in `accumulator.v` is the main 8-bit working register. LDA loads RAM data into it. ADD, SUB, MUL, and DIV use it as the left ALU input, then write the ALU result back into it during `ALU_WRITE`.

```
1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         data_out <= 8'h00;
4     else if (load)
5         data_out <= data_in;
6 end
```

Listing 5: Accumulator load logic from `accumulator.v`

3.6 B Register

The B Register in `b_register.v` stores the second ALU operand. Arithmetic instructions first place `RAM[operand]` on the bus and load it into B. On the next state, the ALU combines the accumulator and B register according to the opcode.

```
1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         data_out <= 8'h00;
4     else if (load)
5         data_out <= data_in;
6 end
```

Listing 6: B Register load logic from `b_register.v`

3.7 ALU

The ALU in `alu.v` is combinational. It receives the accumulator value, B register value, and opcode. It continuously computes a result, but the result only changes CPU state when the control unit writes it back to the accumulator.

```
1 module alu(
2     input [7:0] a,
3     input [7:0] b,
4     input [3:0] opcode,
5     output reg [7:0] result,
6     output reg div_zero
7 );
```

Listing 7: ALU module interface from `alu.v`

3.8 Output Register

The Output Register in `output_register.v` stores the visible CPU result. When the OUT instruction executes, the control unit selects the accumulator onto the bus and asserts `out_load`. The output register then holds that value even as later internal datapath values change.

```
1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         data_out <= 8'h00;
4     else if (load)
5         data_out <= data_in;
6 end
```

Listing 8: Output Register load logic from `output_register.v`

3.9 Control Unit

The Control Unit in `control_unit.v` is the FSM that sequences the datapath. It selects the bus source, enables register loads, increments the PC, chooses the ALU opcode, and asserts the halted output when HLT is reached.

```
1  FETCH_ADDR: begin
2      bus_select = BUS_PC;
3      mar_load = 1'b1;
4      next_state = FETCH_INST;
5  end
6
7  FETCH_INST: begin
8      bus_select = BUS_RAM;
9      ir_load = 1'b1;
10     pc_increment = 1'b1;
11     next_state = DECODE;
12 end
13
14 DECODE: begin
15     case (opcode)
16     OP_NOP:
17         next_state = FETCH_ADDR;
18     OP_HLT:
19         next_state = HALT_STATE;
20     OP_OUT:
21         next_state = OUT_WRITE;
22     OP_LDA, OP_ADD, OP_SUB, OP_MUL, OP_DIV:
23         next_state = MEM_ADDR;
24     default:
25         next_state = FETCH_ADDR;
26     endcase
27 end
```

Listing 9: Fetch and decode control from `control_unit.v`

3.10 Bus Multiplexer

The Bus Multiplexer in `bus_mux.v` implements the shared 8-bit datapath. It selects PC, RAM, instruction operand, accumulator, ALU result, or zero. Four-bit values such as the PC and operand are zero-extended before entering the 8-bit bus. This module is the FPGA-friendly replacement for a textbook tri-state SAP-1 bus.

```
1  case (select)
2      BUS_PC:
3          bus_data = {4'h0, pc_data};
4      BUS_RAM:
5          bus_data = ram_data;
6      BUS_OPERAND:
7          bus_data = {4'h0, ir_operand};
8      BUS_ACC:
9          bus_data = acc_data;
10     BUS_ALU:
11         bus_data = alu_data;
12     default:
13         bus_data = 8'h00;
14 endcase
```

Listing 10: Bus source selection from `bus_mux.v`

4 Instruction Set and Opcode Format

Each instruction is 8 bits. The upper 4 bits store the opcode, and the lower 4 bits store the operand address.

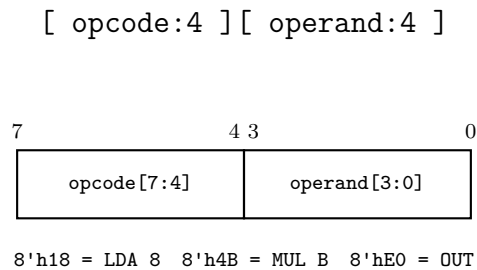


Figure 2: Eight-bit instruction format used by the modified SAP-1.

Opcode	Instruction	Description	Register Transfer
0	NOP	No operation	None
1	LDA addr	Load memory value into A	A ← RAM[addr]
2	ADD addr	Add memory value to A	A ← A + RAM[addr]
3	SUB addr	Subtract memory value from A	A ← A - RAM[addr]
4	MUL addr	Multiply A by memory value	A ← low8(product)
5	DIV addr	Divide A by memory value	A ← quotient
E	OUT	Copy A to output register	OUT ← A
F	HLT	Halt execution	halted ← 1

Example encodings are shown below.

Assembly	Machine Code	Meaning
LDA 8	8'h18	Load RAM[8] into A.
ADD 9	8'h29	Add RAM[9] to A.
SUB A	8'h3A	Subtract RAM[A] from A.
MUL B	8'h4B	Multiply A by RAM[B].
DIV C	8'h5C	Divide A by RAM[C].
OUT	8'hE0	Copy A to the output register.
HLT	8'hF0	Stop execution.

5 ALU Design

The ALU receives the accumulator value **a**, the B register value **b**, and the 4-bit opcode. ADD and SUB use normal 8-bit arithmetic. Overflow and underflow wrap naturally because the CPU stores only the lower 8 bits in the accumulator. MUL and DIV are implemented manually; MUL does not use Verilog `*` and DIV does not use Verilog `/`. This keeps the arithmetic behavior simple and consistent with the 8-bit SAP-1 datapath.

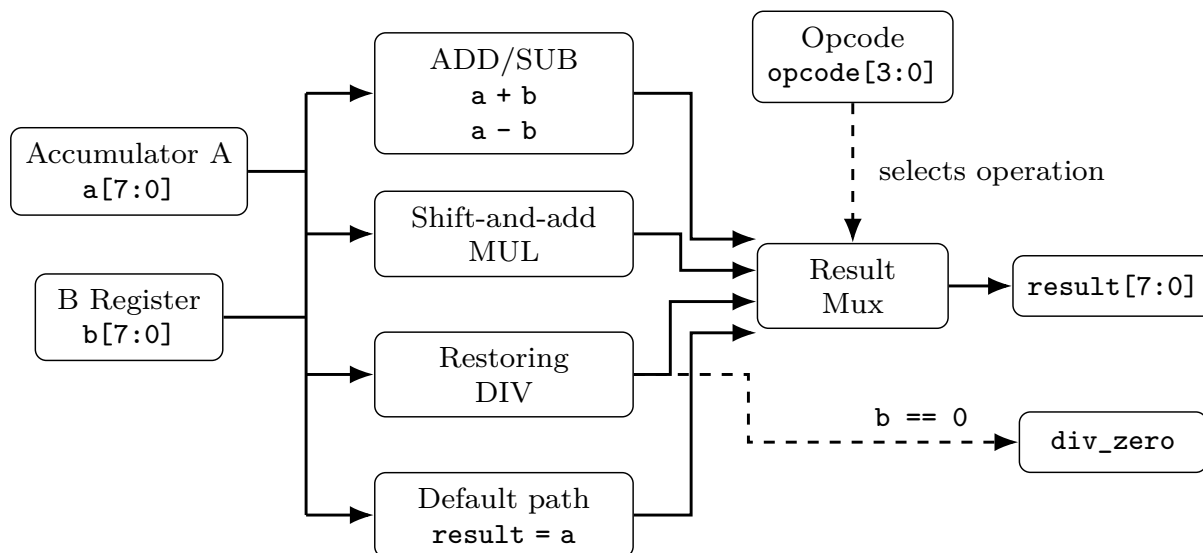


Figure 3: Internal ALU structure showing opcode-based selection of ADD, SUB, MUL, DIV, and divide-by-zero flag generation.

5.1 ADD and SUB

ADD computes $a + b$. SUB computes $a - b$. Both operations are unsigned 8-bit operations, so values wrap around when the mathematical result is outside the range 0 to 255.

```

1 case (opcode)
2   OP_ADD:
3     result = a + b;
4   OP_SUB:
5     result = a - b;
6 endcase
  
```

Listing 11: ADD and SUB operation selection from alu.v

5.2 Multiplication

MUL uses a fixed 8-iteration shift-and-add algorithm. The ALU starts with a 16-bit partial product equal to zero and a 16-bit shifted copy of the left operand. For each bit of the multiplier, it adds the shifted value into the partial product only if that multiplier bit is one. The shifted value moves left after every iteration. The ALU returns `partial[7:0]`, so multiplication overflow keeps the lower 8 bits.

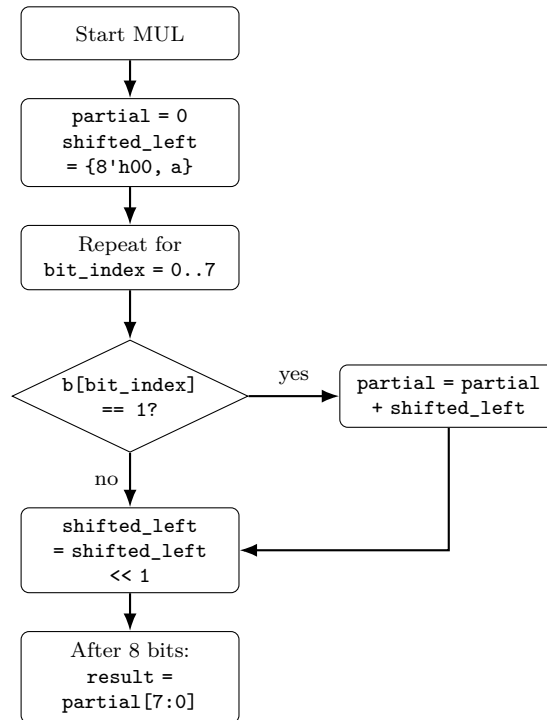


Figure 4: Shift-and-add flow used for MUL without Verilog multiplication.

```
1 partial = 16'h0000;
2 shifted_left = {8'h00, left_value};
3
4 for (bit_index = 0; bit_index < 8; bit_index = bit_index + 1) begin
5     if (right_value[bit_index])
6         partial = partial + shifted_left;
7     shifted_left = shifted_left << 1;
8 end
9
10 shift_add_product = partial[7:0];
```

Listing 12: Shift-and-add multiplication from alu.v

5.3 Division

DIV uses restoring division. The quotient and remainder start at zero. The algorithm processes dividend bits from bit 7 down to bit 0, shifts each bit into the partial remainder, compares the remainder with the divisor, subtracts the divisor when possible, and sets the corresponding quotient bit. The remainder is discarded, so DIV produces an integer quotient.

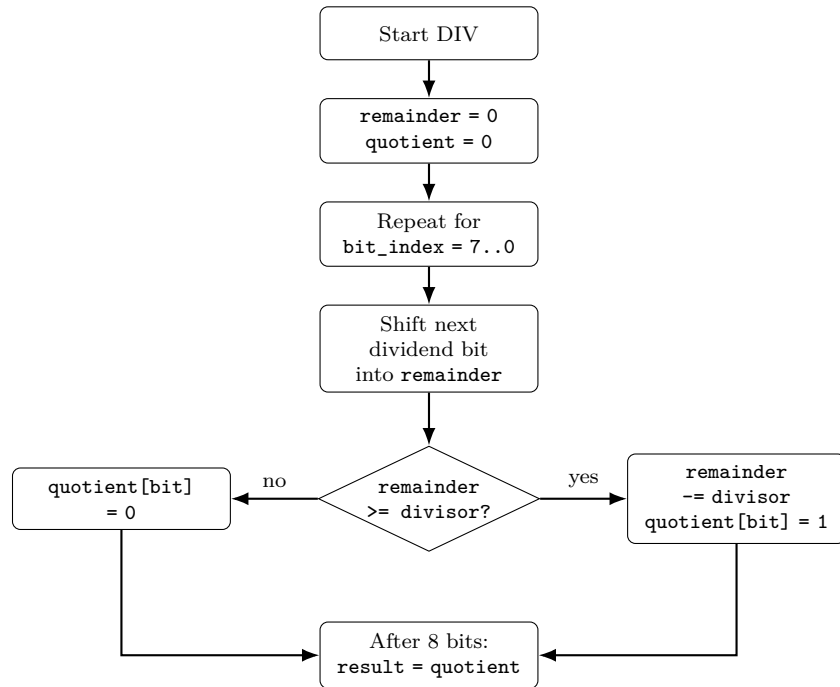


Figure 5: Restoring division flow used for DIV without Verilog division.

```
1 remainder = 9'h000;
2 quotient = 8'h00;
3
4 for (bit_index = 7; bit_index >= 0; bit_index = bit_index - 1) begin
5     remainder = {remainder[7:0], dividend[bit_index]};
6     if (remainder >= {1'b0, divisor}) begin
7         remainder = remainder - {1'b0, divisor};
8         quotient[bit_index] = 1'b1;
9     end
10 end
11
12 restoring_quotient = quotient;
```

Listing 13: Restoring division quotient from alu.v

5.3.1 Division by Zero Handling

Division by zero is handled in the ALU and then latched at the CPU level. If the divisor input is zero during a DIV operation, the ALU produces 8'hFF and asserts its raw combinational `div_zero` output.

```
1 if (b == 8'h00) begin
2     result = 8'hFF;
3     div_zero = 1'b1;
4 end else begin
5     result = restoring_quotient(a, b);
6 end
```

Listing 14: ALU divide-by-zero result from alu.v

The CPU-level `div_zero` flag in `sap1_top.v` is updated only when the ALU result is committed into the accumulator. This matters because the ALU is combinational: its raw flag can change whenever inputs change, but the CPU should record a divide-by-zero event only when a DIV instruction actually writes back.

```
1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         div_zero <= 1'b0;
4     else if (acc_load && bus_select == BUS_ALU) begin
5         if (alu_opcode == OP_DIV)
6             div_zero <= alu_div_zero;
7         else
8             div_zero <= 1'b0;
9     end
10 end
```

Listing 15: CPU-level `div_zero` latch from sap1_top.v

5.4 Opcode-Based Result Selection

The final ALU result is selected using the opcode. Unsupported opcodes pass the accumulator value through unchanged.

```
1 always @(a or b or opcode) begin
2     div_zero = 1'b0;
3
4     case (opcode)
5         OP_ADD:
6             result = a + b;
7         OP_SUB:
8             result = a - b;
9         OP_MUL:
10            result = shift_add_product(a, b);
11         OP_DIV: begin
12             if (b == 8'h00) begin
13                 result = 8'hFF;
14                 div_zero = 1'b1;
15             end else begin
16                 result = restoring_quotient(a, b);
17             end
18         end
19         default:
20             result = a;
21     endcase
22 end
```

Listing 16: ALU operation selection from alu.v

6 Control Unit and FSM Design

The control unit is a finite state machine that coordinates the fetch, decode, and execute cycle. It selects the bus source, enables registers, increments the program counter, selects the ALU operation, and asserts the halted signal. Each state performs one small register-transfer step, which makes execution timing clear in simulation.

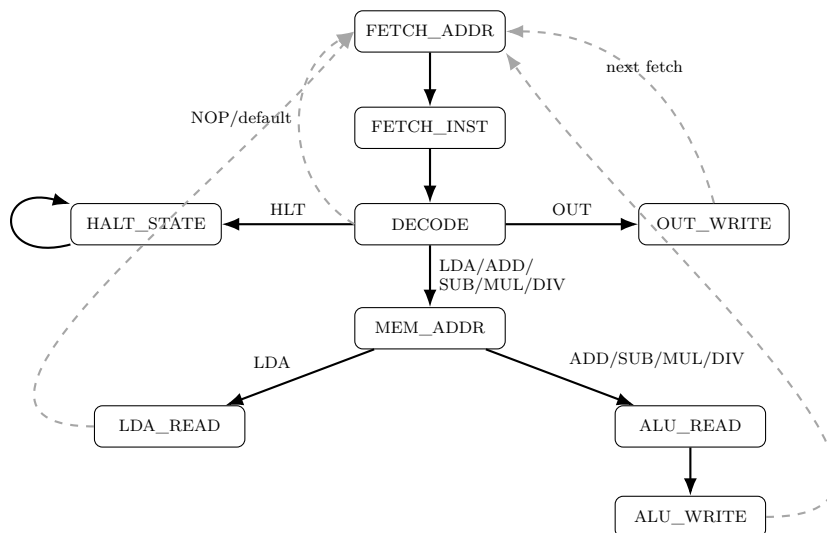


Figure 6: FSM state diagram of the modified SAP-1 control unit.

State	Purpose	Action
FETCH_ADDR	Address instruction memory	Put PC on bus and load MAR.
FETCH_INST	Fetch instruction	Put RAM data on bus, load IR, and increment PC.
DECODE	Decode opcode	Choose the next execute state.
MEM_ADDR	Address operand memory	Put instruction operand on bus and load MAR.
LDA_READ	Execute LDA	Put RAM data on bus and load A.
ALU_READ	Read ALU operand	Put RAM data on bus and load B.
ALU_WRITE	Commit ALU result	Put ALU result on bus and load A.
OUT_WRITE	Execute OUT	Put A on bus and load output register.
HALT_STATE	Stop execution	Assert halted and remain in the halt state.

The fetch sequence is common to every instruction:

```

FETCH_ADDR: MAR ← PC
FETCH_INST: IR ← RAM[MAR], PC ← PC + 1
DECODE: inspect opcode
  
```

LDA then executes by loading the operand address into MAR and reading RAM into the accumulator. Arithmetic instructions load the memory operand into the B register, then write the ALU result back into the accumulator. OUT copies the accumulator into the output register. HLT enters a stable halt state, preventing later instructions from executing.

7 Simulation Results

The self-checking testbench loads programs into RAM using the top-level programming interface, releases reset, waits for the processor to halt, and checks the output register and divide-by-zero flag.

7.1 Main Program

The normal test program computes:

$$((10 + 3 - 4) \times 2) / 3 = 6$$

```

1 write_memory(4'h0, 8'h18); // LDA RAM[0x8]
2 write_memory(4'h1, 8'h29); // ADD RAM[0x9]
3 write_memory(4'h2, 8'h3A); // SUB RAM[0xA]
4 write_memory(4'h3, 8'h4B); // MUL RAM[0xB]
5 write_memory(4'h4, 8'h5C); // DIV RAM[0xC]
6 write_memory(4'h5, 8'hE0); // OUT
7 write_memory(4'h6, 8'hF0); // HLT
8 write_memory(4'h8, 8'd10);
9 write_memory(4'h9, 8'd3);
10 write_memory(4'hA, 8'd4);
11 write_memory(4'hB, 8'd2);
12 write_memory(4'hC, 8'd3);

```

Listing 17: Main program loaded in sap1_tb.v

Address	Value	Meaning
0	18	LDA 8
1	29	ADD 9
2	3A	SUB A
3	4B	MUL B
4	5C	DIV C
5	E0	OUT
6	F0	HLT
8	0A	Data 10
9	03	Data 3
A	04	Data 4
B	02	Data 2
C	03	Data 3

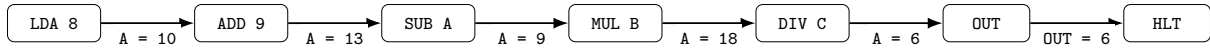


Figure 7: Normal test program execution trace.

7.2 Waveform Walk-Through

The normal waveform shows the full instruction sequence. First, `LDA 8` loads RAM address 8, which contains decimal 10, into the accumulator. Next, `ADD 9` loads RAM address 9 into the B register and adds 3, so the accumulator becomes 13. Then `SUB A` subtracts RAM address A, which contains 4, so the accumulator becomes 9.

The next arithmetic instructions use the same memory-operand path. `MUL B` multiplies the accumulator by RAM address B, which contains 2, so the accumulator becomes 18. `DIV C` divides 18 by RAM address C, which contains 3, so the accumulator becomes 6. Finally, `OUT` copies 6 into the output register, and `HLT` asserts `halted` and stops the control FSM.

The bus and ALU result signals can change between states because the bus multiplexer and ALU are combinational. The lasting architectural updates occur only when the control unit asserts the appropriate register load signal on a clock edge.

7.3 Divide-by-Zero Result

The divide-by-zero test loads 25 into the accumulator and then executes `DIV` with a divisor stored as zero in RAM. During the `DIV` operation, the ALU detects `b == 8'h00`, returns `8'hFF`, and asserts `raw_alu_div_zero`. When the `DIV` result is committed in `sap1_top.v`, the CPU-level `div_zero` output becomes 1. The `OUT` instruction then copies `8'hFF` to the output register.

7.4 Test Summary

Test Case	Expected Behavior	Result
Normal arithmetic flow	Output is <code>8'h06</code> , <code>div_zero</code> = 0.	Pass
Division by zero	Output is <code>8'hFF</code> , <code>div_zero</code> = 1.	Pass
Multiplication overflow	<code>20 * 20</code> keeps lower 8 bits, output <code>8'h90</code> .	Pass
Addition overflow	<code>250 + 10</code> wraps to <code>8'h04</code> .	Pass
Subtraction underflow	<code>3 - 10</code> wraps to <code>8'hF9</code> .	Pass
Division truncation	<code>7 DIV 2</code> outputs <code>8'h03</code> .	Pass
LDA overwrite	Second LDA replaces the previous accumulator value.	Pass
HLT behavior	Instructions after HLT do not execute, and output remains stable.	Pass

```

run all

Running test: normal LDA ADD SUB MUL DIV
PASS: normal LDA ADD SUB MUL DIV out=6 0x06.
PASS: normal LDA ADD SUB MUL DIV div_zero=0.

Running test: division by zero
PASS: division by zero out=255 0xff.
PASS: division by zero div_zero=1.

Running test: multiplication overflow
PASS: multiplication overflow out=144 0x90.
PASS: multiplication overflow div_zero=0.

Running test: addition overflow
PASS: addition overflow out=4 0x04.
PASS: addition overflow div_zero=0.

Running test: subtraction underflow
PASS: subtraction underflow out=249 0xf9.
PASS: subtraction underflow div_zero=0.

Running test: division truncates remainder
PASS: division truncates remainder out=3 0x03.
PASS: division truncates remainder div_zero=0.

Running test: LDA overwrite
PASS: LDA overwrite out=99 0x63.
PASS: LDA overwrite div_zero=0.

Running test: HLT stops execution
PASS: HLT stops execution out=5 0x05.
PASS: HLT stops execution div_zero=0.
PASS: HLT holds out and halted stable.
ALL TESTS PASSED
$finish called at time : 5656 ns : File "D:/Project/modified-sap-1/modified-sap-1.srscs/sim_1/new/sap1_tb.v" Line 246

```

Figure 8: Vivado Tcl Console showing the self-checking testbench results.

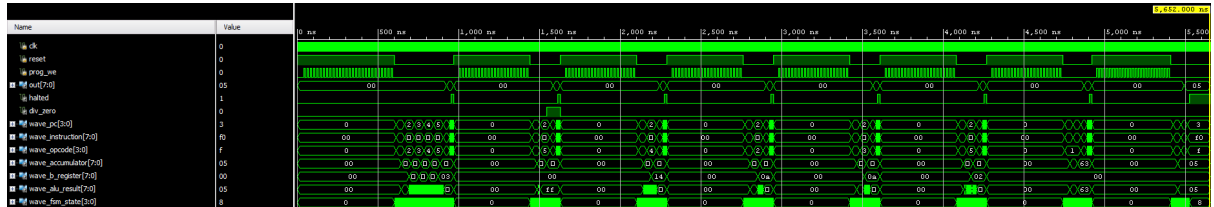


Figure 9: Vivado waveform showing SAP-1 datapath and control signals during simulation.

8 Conclusion

The modified SAP-1 computer was implemented in Verilog HDL with the required processor components and an FSM-based fetch, decode, and execute cycle. The instruction set includes LDA, ADD, SUB, MUL, DIV, OUT, and HLT. The ALU supports multiplication and division using manual algorithms, and division by zero is handled by returning 8'hFF and setting the `div_zero` flag.

Behavioral simulation in Vivado/XSim verified normal instruction execution and edge cases. The self-checking testbench completed successfully and reported that all tests passed.

9 References

1. Karen Ok, “Designing and Implementing a SAP-1 Computer,” GitHub Pages. Available: <https://karenok.github.io/SAP-1-Computer/>